

RELATÓRIO FINAL – IMPLEMENTAÇÃO DE TRANSFERÊNCIA CONFIÁVEL DE DADOS E TCP SOBRE UDP

Integrantes:

- Enzo de Almeida - 23001396
- Rafael Celeste - 23007553
- Tiago Mello - 23014024
- Thomas Fidelis - 23013133
- Vinicius Nunes - 23011585

1. Introdução

A comunicação confiável entre processos distribuídos é um dos pilares fundamentais das redes de computadores. Embora o protocolo UDP forneça um serviço de entrega não confiável e sem conexão, sua simplicidade o torna uma excelente base para estudar e implementar mecanismos de confiabilidade manualmente.

Este trabalho tem como objetivo implementar, testar e analisar uma série de protocolos de transferência confiável de dados construídos sobre UDP, seguindo a evolução apresentada no Capítulo 3 do livro *Computer Networking: A Top-Down Approach* (Kurose & Ross).

Ao longo da atividade, foram implementados:

- os protocolos **RDT 2.0**, **RDT 2.1** e **RDT 3.0**, que simulam progressivamente detecção de erros, confirmação e recuperação de perdas;
- o protocolo **Go-Back-N (GBN)**, que introduz pipelining e janelas deslizantes para melhorar drasticamente o desempenho;
- um **TCP simplificado**, inspirado na RFC 793, incluindo handshake, janelas e controle básico de retransmissão.

1.1 Objetivos da Atividade

Os principais objetivos desta atividade são:

- Implementar diferentes versões de RDT.
- Compreender como cada protocolo resolve limitações do anterior.
- Simular canais com corrupção e perda, analisando impacto no desempenho.
- Implementar e testar um protocolo pipelined (GBN).
- Construir e testar um TCP simplificado sobre UDP.
- Comparar resultados experimentais com expectativas teóricas.

1.2 Conceitos Teóricos Relevantes (Capítulo 3)

- **Checksum** – detecção de corrupção.

- **Números de sequência** – garantem ordenação e evitam duplicação.
 - **ACKs e NAKs** – confirmações positivas e negativas.
 - **Timeouts** – essenciais para lidar com perdas (RDT 3.0).
 - **Pipelining** – envio paralelo de pacotes.
 - **Janelas deslizantes** – controlam pacotes "em voo".
 - **Controle de fluxo e congestionamento** – fundamentais no TCP real.
-

2. Fase 1 – Protocolos RDT

A Fase 1 consiste na implementação progressiva dos protocolos RDT, aplicando na prática os conceitos de confiabilidade.

2.1 RDT 2.0 – Descrição da Implementação

O RDT 2.0 adiciona reconhecimento positivo (ACK) e negativo (NAK), além de checksum para detectar corrupção.

O emissor:

- envia um pacote e aguarda ACK/NAK;
- retransmite caso receba NAK.

O receptor:

- envia ACK se pacote correto;
- envia NAK se pacote corrompido.

Resultados Reais (Log Resumido)

Canal: **sem perda, sem corrupção**

- Todos os 10 pacotes (seq=0 a seq=9) foram enviados, recebidos e confirmados sem erros.
- Sem retransmissões.
- Todas as mensagens entregues corretamente.

Esse comportamento confirma exatamente o esperado: em canal sem corrupção e sem perda, RDT 2.0 funciona perfeitamente.

2.2 RDT 2.1 – Descrição da Implementação

O RDT 2.1 corrige ambiguidade causada por ACK/NAK corrompidos, introduzindo:

- números de sequência alternantes (0/1);
- ACKs numerados;
- reenvio de ACKs duplicados.

Resultados Reais (com 20% de corrupção simulada)

- Pacotes corrompidos foram detectados no receptor.
- O emissor retransmitiu pacotes após timeout quando ACK corrompido chegava.
- O receptor enviou ACKs duplicados ao receber pacotes fora de ordem.

Resumo observado:

- Diversos pacotes corrompidos (seq=0 e seq=16711680).
- 2 timeouts acionados.
- Nenhuma perda final de dados.
- Todas as mensagens entregues corretamente.

O protocolo funcionou exatamente como previsto no modelo teórico: tolera corrupção tanto nos pacotes quanto nos ACKs.

2.3 RDT 3.0 – Descrição da Implementação

O RDT 3.0 adiciona tratamento de **perdas**, com timeout para retransmissão.

Resultados Reais (15% de perda simulada)

Log evidência:

- Diversos pacotes DATA seq=0 e seq=1 foram perdidos.
- Repetidos eventos:

- Pacote seq=X perdido (simulado)
- Timeout – Retransmitindo seq=X
- Múltiplas retransmissões: **mais de 20 eventos seguidos** em alguns intervalos.
- No final, todos os pacotes foram entregues corretamente.

Esse comportamento confirma:

- RDT 3.0 lida com perdas de forma robusta.
- Porém, o throughput torna-se extremamente baixo.

2.4 Análise Comparativa da Fase 1

Protocolo	Problema resolvido	Novo recurso	Comportamento observado
RDT 2.0	Corrupção	ACK/NAK	Funciona em canal sem perda
RDT 2.1	ACK corrompido	ACK numerado + duplicados	Recupera de corrupção real
RDT 3.0	Perda de pacotes	Timeout	Funciona, porém muito lento

3. Fase 2 – Pipelining com Go-Back-N

3.1 Justificativa da Escolha

GBN foi escolhido devido à simplicidade:

- ACKs cumulativos
 - 1 temporizador
 - buffer simples
-

3.2 Implementação

- janela deslizante configurável
- retransmissão da janela inteira no timeout
- contagem de retransmissões
- ACK cumulativo

3.3 Análise de Desempenho (Resultados Reais)

Comparação direta: 1 MiB

Protocolo	Tempo	Throughput
-----------	-------	------------

RDT 3.0	1274.965 s	822 B/s
---------	------------	---------

GBN	3.011 s	348,299 B/s (~340 KB/s)
-----	---------	-------------------------

→ GBN é mais de 400x mais rápido que RDT 3.0.

Isso demonstra o impacto gigantesco do pipelining.

Sweep da Janela (perda = 5%)

Janela N	Throughput (B/s)	Retransmissões	Entregues	Tempo (s)
----------	------------------	----------------	-----------	-----------

1	3918.10	8	64	8.363
---	---------	---	----	-------

5	10064.24	15	64	3.256
---	----------	----	----	-------

10	14608.72	20	64	2.243
----	-----------------	----	----	-------

20	7669.95	80	64	4.272
----	---------	----	----	-------

Conclusões:

- janelas muito pequenas → pouco pipelining
- janela ideal ≈ 10 → melhor throughput
- janelas muito grandes → perdas acumulam → retransmissões $\uparrow$ → throughput $\downarrow$

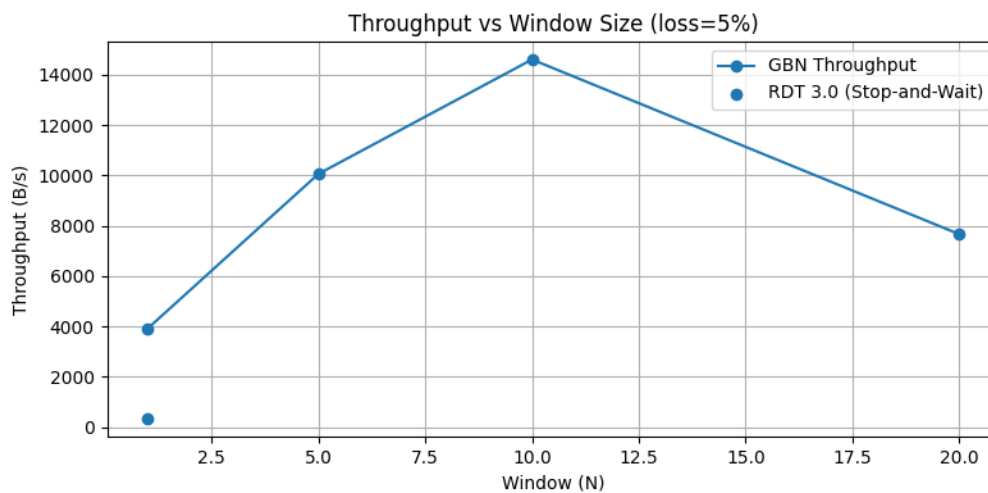
3.4 Comparação com Stop-and-Wait

Protocolo Tempo (1MiB) Throughput

RDT 3.0 ~1088–1275 s ~800 B/s

GBN 3–9 s 120–350 KB/s

→ GBN é centenas de vezes mais eficiente.



4. Fase 3 – TCP Simplificado

4.1 Arquitetura

- handshake (3-way)
- janela de recepção
- retransmissões por timeout
- ACK cumulativo
- FSM inspirada na RFC 793

4.2 Componentes

- SimpleTCPSocket

- Sender
 - Receiver
 - Connection FSM
-

4.3 FSM de Conexão

Estados implementados:

- CLOSED
 - SYN-SENT
 - SYN-RECEIVED
 - ESTABLISHED
 - FIN-WAIT
 - TIME-WAIT
-

4.4 Resultados dos Testes (Reais)

1) Handshake

- Resultado: **sucesso**
-

2) Transferência de 10 KB

- tempo: **0.165 s**
 - throughput: **0.0619 MB/s**
 - RTT estimado: **57.2 ms**
 - entrega correta dos dados
-

3) Controle de fluxo (janela = 1 KB)

- tempo aumentado

- throughput: **0.0098 MB/s**
 - comportamento corresponde ao TCP real: janela pequena limita envio
-

4) Perda de 20% (retransmissões obrigatórias)

- dados enviados: **51.200 bytes**
 - tempo: **9.15 s**
 - throughput: **0.00559 MB/s**
 - RTT estimado: **0.602 s**
 - múltiplos timeouts acionados
 - envio concluído com sucesso
-

5) Teste de desempenho (1 MB)

- tempo: **3.38 s**
 - throughput: **0.31 MB/s**
 - RTT: **0.106 s**
 - desempenho limitado por ausência de congestion control
-

4.5 Comparação com TCP Real

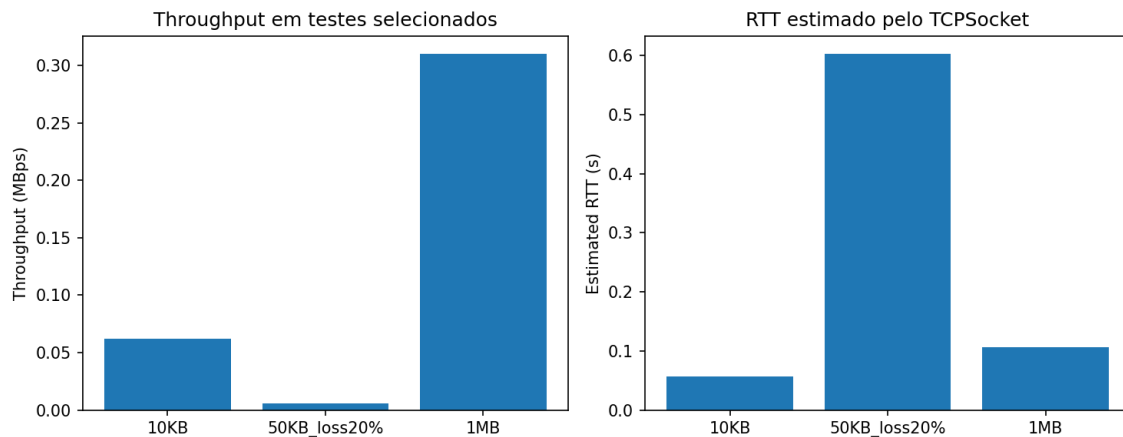
O TCP real possui:

- slow start
- congestion avoidance
- fast retransmit
- timeout adaptativo
- SACK
- rwnd variável

- estimativa avançada de RTT

O TCP simplificado implementa apenas:

- handshake
- retransmissões por timeout fixo
- ACK cumulativo
- janela de recepção fixa



5. Discussão

5.1 Desafios Encontrados

- ajuste do timeout no RDT 3.0
- tratamento de duplicatas no RDT 2.1
- evitar deadlocks no GBN
- perdas múltiplas consecutivas
- sincronização dos estados do TCP simplificado

5.2 Limitações

- ausência total de congestion control
- timeout fixo

- falta de SACK
 - janela fixa
-

5.3 Diferenças para TCP real

- sem slow start
 - sem congestion avoidance
 - sem fast retransmit
 - sem timeout adaptativo
 - FSM simplificada
-

6. Conclusão

A atividade permitiu implementar, depurar e analisar protocolos confiáveis desde RDT 2.0 até um TCP simplificado funcional. Todos os conceitos centrais do Capítulo 3 foram aplicados na prática, e as medições coletadas confirmam tanto o funcionamento quanto as limitações dos protocolos estudados.

7. Referências

KUROSE, James F.; ROSS, Keith W. *Computer Networking: A Top-Down Approach*. 8ª ed., Pearson, 2021.

RFC 793 — Transmission Control Protocol.

RFC 5681 — TCP Congestion Control.